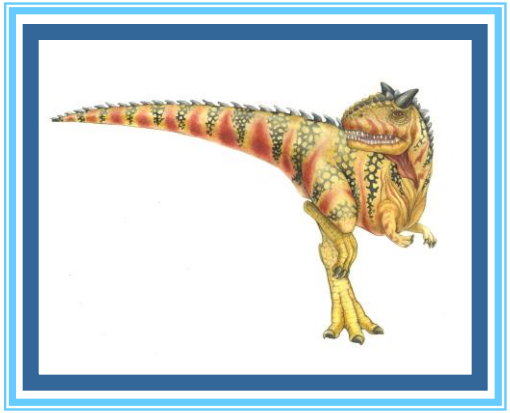
Project 1: Thread

OS lab 3

Chijin Zhou
East China Normal University

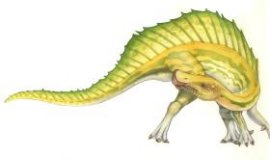




Lab3 Overview

- Lab1 基础Linux使用 （15%）
- Lab2 PintOS 环境搭建（15%）
- **Lab3 线程管理 （35%）**
- Lab4 实现系统调用（35%）

- 注：补交分数打6折。
- Lab3的截止日期是12月8日。
- Lab4的截止时间是12月30日。



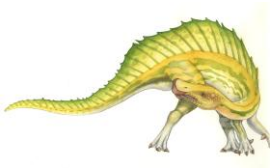


Lab3 Overview

我们需要将 make check 中的所有测试用例都通过

```
pass tests/threads/mlfqs-block
pass tests/threads/alarm-single
pass tests/threads/alarm-multiple
pass tests/threads/alarm-simultaneous
pass tests/threads/alarm-priority
pass tests/threads/alarm-zero
pass tests/threads/alarm-negative
pass tests/threads/priority-change
pass tests/threads/priority-donate-one
pass tests/threads/priority-donate-multiple
pass tests/threads/priority-donate-multiple2
pass tests/threads/priority-donate-nest
pass tests/threads/priority-donate-sema
pass tests/threads/priority-donate-lower
pass tests/threads/priority-fifo
pass tests/threads/priority-preempt
pass tests/threads/priority-sema
pass tests/threads/priority-condvar
pass tests/threads/priority-donate-chain
pass tests/threads/mlfqs-load-1
pass tests/threads/mlfqs-load-60
pass tests/threads/mlfqs-load-avg
pass tests/threads/mlfqs-recent-1
pass tests/threads/mlfqs-fair-2
pass tests/threads/mlfqs-fair-20
pass tests/threads/mlfqs-nice-2
pass tests/threads/mlfqs-nice-10
pass tests/threads/mlfqs-block
All 27 tests passed.
```







How much code will you write?

See pintos.pdf

2.3 FAQ

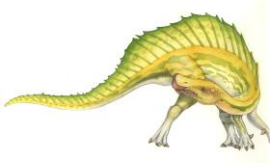
How much code will I need to write?

Here's a summary of our reference solution, produced by the `diffstat` program. The final row gives total lines inserted and deleted; a changed line counts as both an insertion and a deletion.

The reference solution represents just one possible solution. Many other solutions are also possible and many of those differ greatly from the reference solution. Some excellent solutions may not modify all the files modified by the reference solution, and some may modify files not modified by the reference solution.

```
 devices/timer.c         |   42 +++++-
 threads/fixed-point.h   |  120 ++++++++++++++++++
 threads/synch.c         |   88 ++++++++++++-
 threads/thread.c        |  196 ++++++++++++++++++++++++++----
 threads/thread.h        |   23 +++
 5 files changed, 440 insertions(+), 29 deletions(-)
```

'`fixed-point.h`' is a new file added by the reference solution.

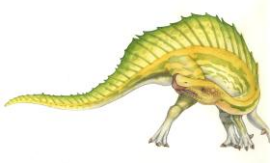




作业3.1 实验报告(25分)

提交实验报告，包括：

- 通过所有测试用例的 make check **截图**
- 说明 alarm-priority 是如何通过的
- 说明 priority-* 是如何通过的
- 说明 mlfqs-* 是如何通过的





作业3.2 上台讲解自己如何实现的(10分)

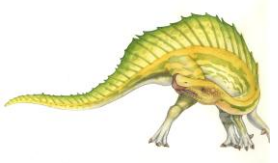
上台讲述自己如何通过测试用例的

- 5分钟ppt讲述 + 1分钟问询
- 11.3、11.17、12.1三周，每周15个名额
- 腾讯文档填写自己名字报名
  - 越早讲，分数越高（decay rate大约是0.7）；
  - 每节课的前一周周二晚上12点固定该节课的报名列表
  - （别改别人的名字，我能看历史操作记录的。。）

2026秋季-lab3报名
扫一扫二维码打开或分享...

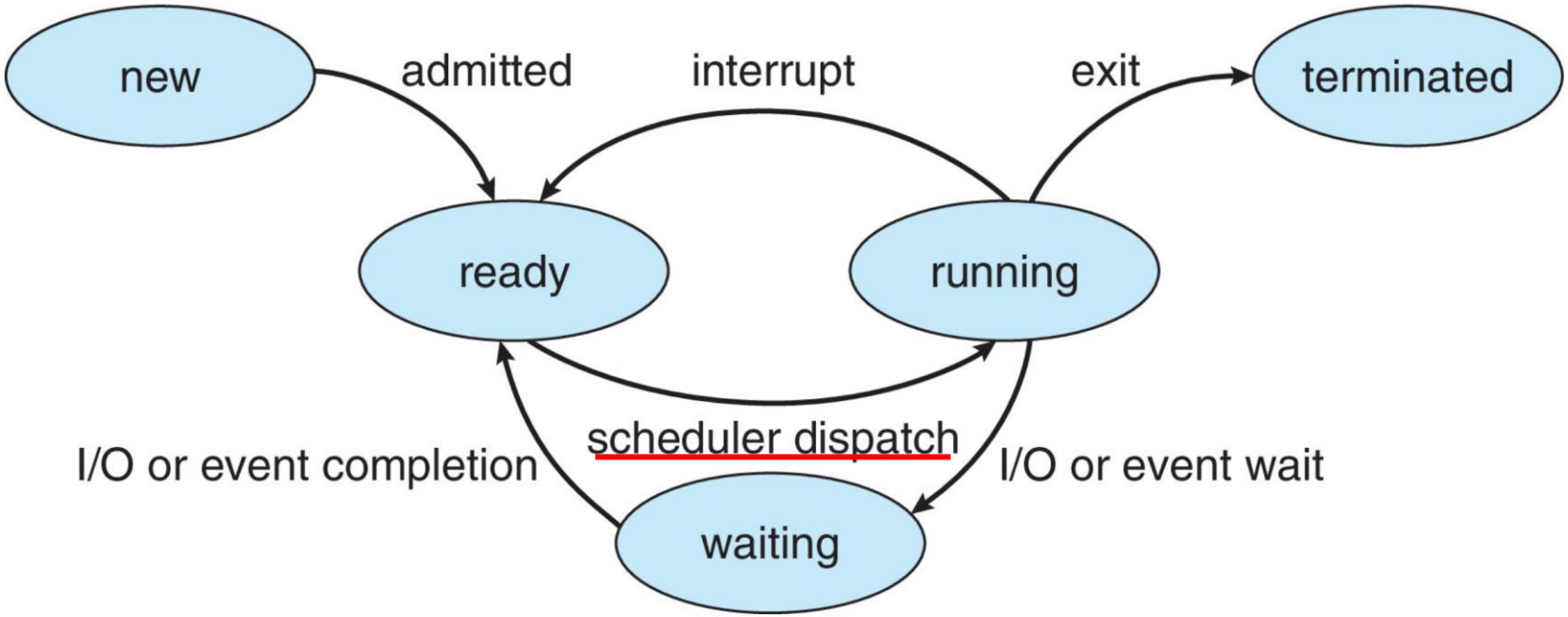
- 腾讯文档 -
可多人实时在线编辑，权限安全可控

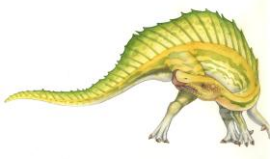
	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O	P
1		第1个讲	第2个讲	第3个讲	第4个讲	第5个讲	第6个讲	第7个讲	第8个讲	第9个讲	第10个讲	第11个讲	第12个讲	第13个讲	第14个讲	第15个讲
2	2026/11/3 (内容：priority)															
3	2026/11/17（内容：mlfqs）															
4	2026/12/1（内容：mlfqs）															





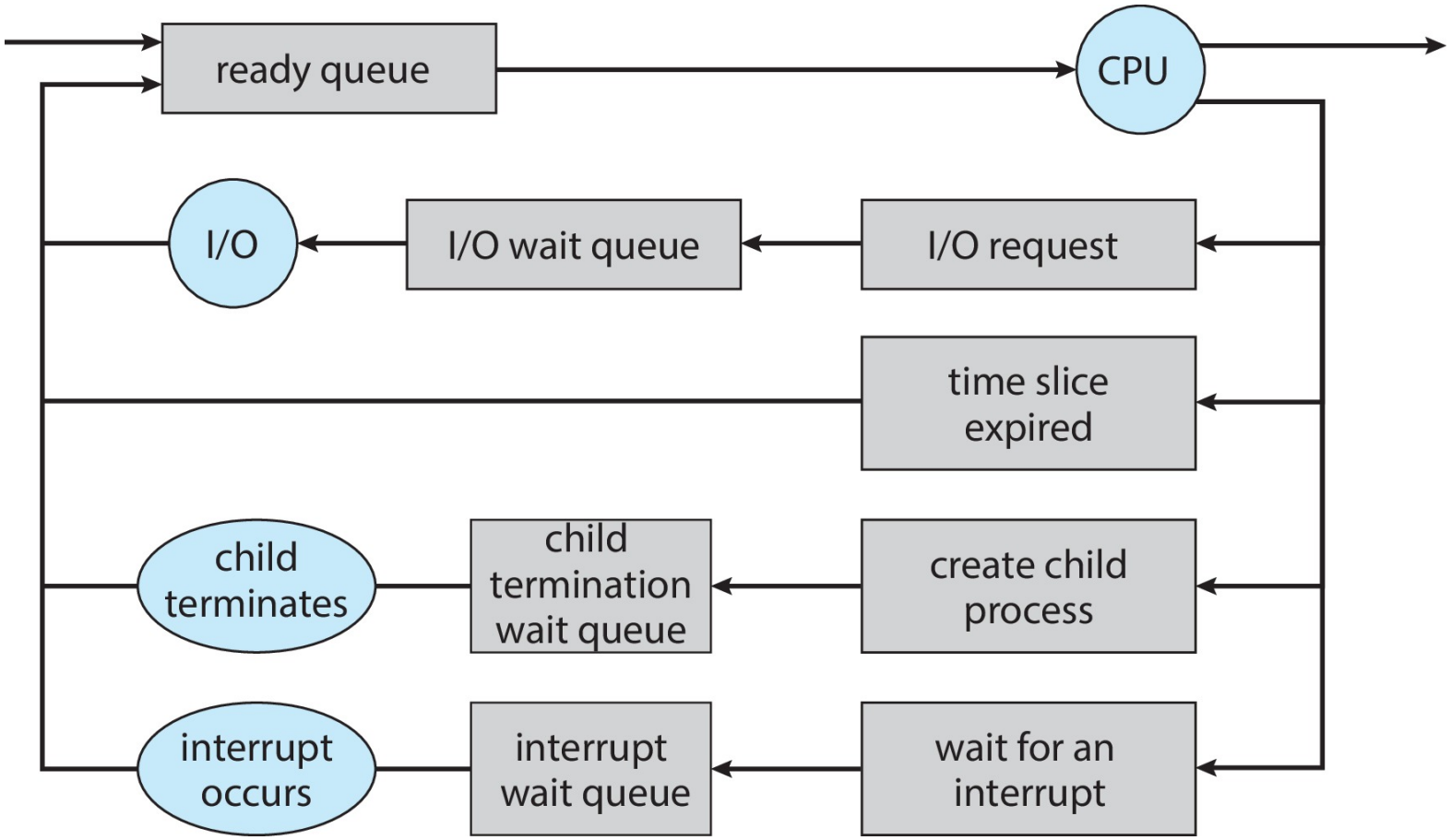
Schedule

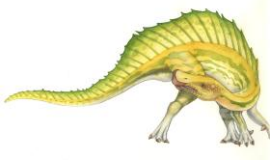

new
admitted
interrupt
exit
terminated
ready
running
scheduler dispatch
I/O or event completion
I/O or event wait
waiting





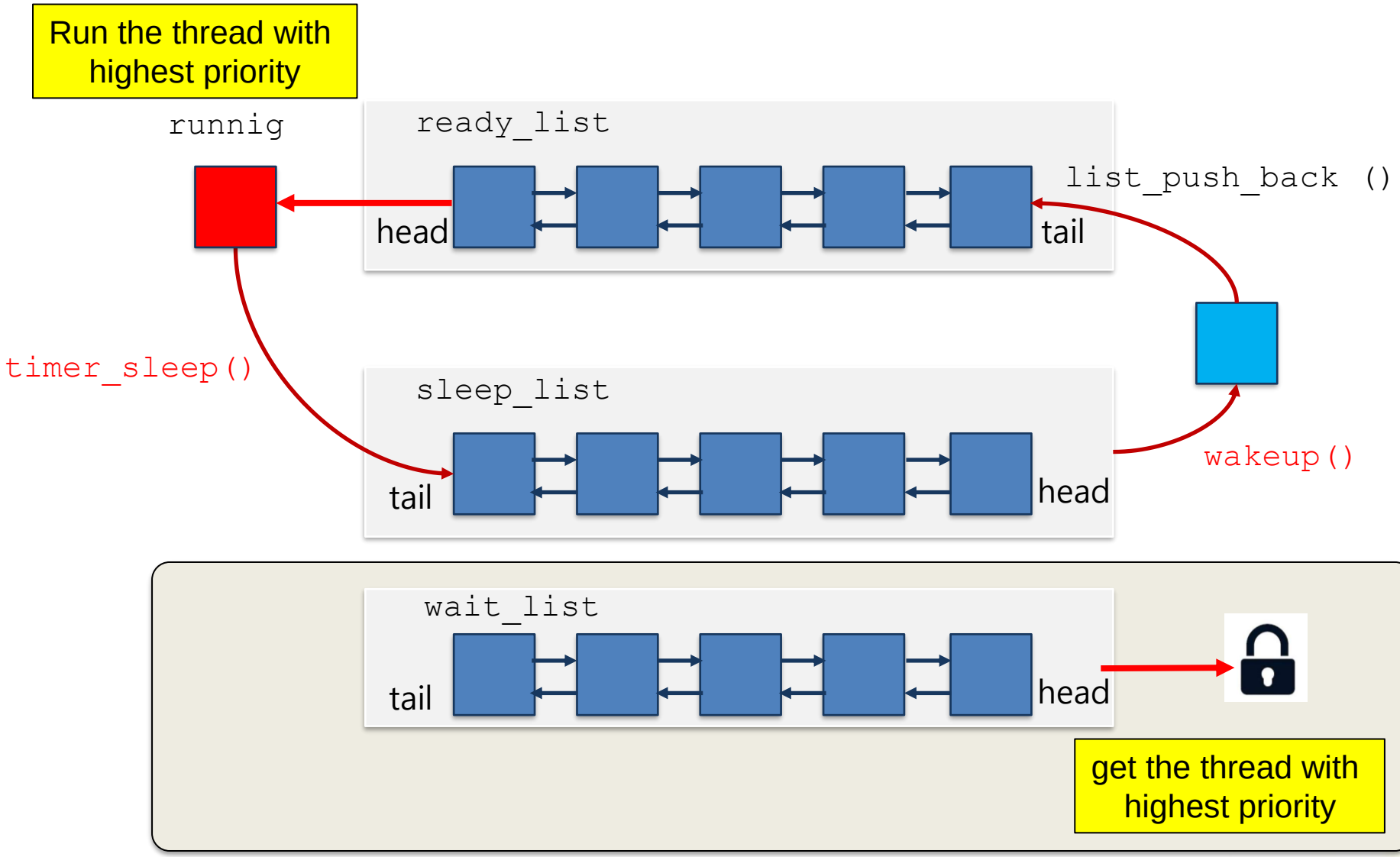
Queue


ready queue
CPU
I/O
I/O wait queue
I/O request
time slice expired
child terminates
child termination wait queue
create child process
interrupt occurs
interrupt wait queue
wait for an interrupt





Priority


Run the thread with highest priority
runnig
ready_list
head
tail
list_push_back ()
timer_sleep()
sleep_list
tail
head
wakeup()
wait_list
tail
head
get the thread with highest priority



Three things to consider

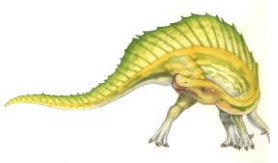
- When selecting a thread to run in the ready list, select the one with the highest priority.
- **Preemption**
  - When inserting the new thread to the ready list, compare the priority with the running thread.
  - Schedule the newly inserted thread if it has the higher priority with the currently running thread.
- **Lock**: semaphore, condition variable,
  - When selecting a thread from the set of threads waiting for a lock (or condition variable), select the one with the highest priority.





Priority in pintos

- Priority ranges from `PRI_MIN(=0)` to `PRI_MAX(=63).`
  - The larger the number, the higher priority.
  - Default is `PRI_DEFAULT(=31)`
- PintOS sets the initial priority when the thread is created by `thread_create()`
- Existing functions
  - `void thread_set_priority (int new_priority)`
    - Change priority of the current thread to `new_priority`
  - `int thread_get_priority (void)`
    - Return priority of the current thread.

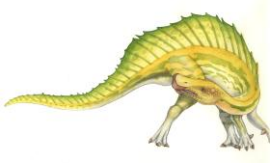




Implementation of Priority Scheduling

```
tid_t thread_create(const char *name, int priority,
                    thread_func *function, void *aux)
```

- Point of updates
  - Insert thread in `ready_list` in the order of priority. (note that it is not scalable)
  - When the thread is added to the `ready_list`, compare priority of new thread and priority of the current thread.
  - If the priority of the new thread is higher, call `schedule()` (the current thread yields CPU).





thread_create()

- When inserting a thread to `ready_list` , compare the priority with the currently running thread.
- If the newly arriving thread has higher priority, preempt the currently running thread and execute the new one.

```
tid_t thread_create (const char *name, int priority,
                     thread_func *function, void *aux)
{
        …
    thread_unblock (t);

    /* compare the priorities of the currently running
thread and the newly inserted one. Yield the CPU if the
newly arriving thread has higher priority*/
    return tid;
}
```



Hint: list_inert_ordered

- `thread_unblock()`
  - When unblocking a thread, use list_inert_ordered instead of list_push_back.
- You are welcome :)

```
void thread_unblock (struct thread *t)
{
   …

    //list_push_back (&ready_list, &t->elem);

    list_insert_ordered(& ready_list, & t-> elem,
                        cmp_priority, NULL);

    t->status = THREAD_READY;
    intr_set_level (old_level);
}
```

delete

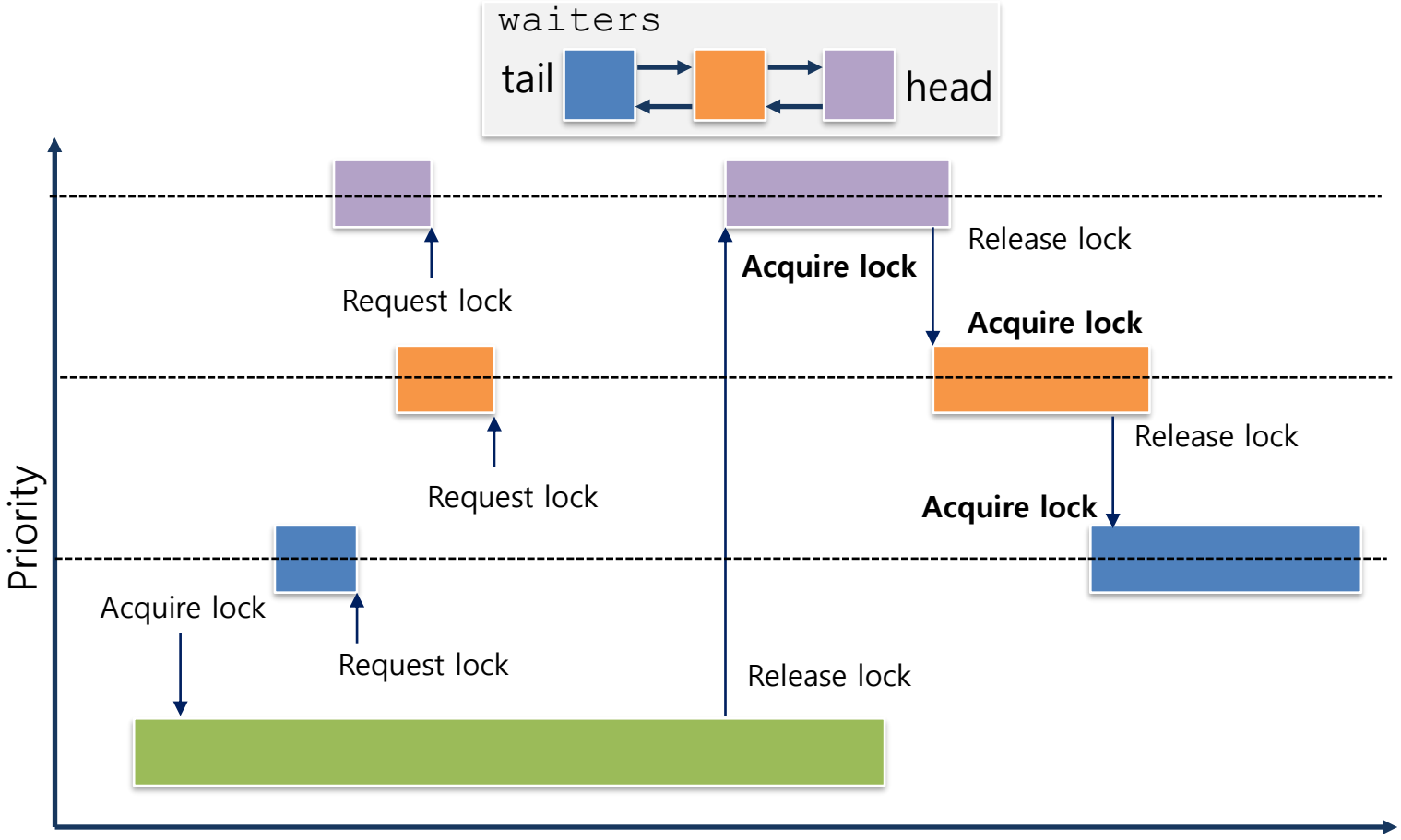
add





Priority-based lock/unlock

- When threads try to acquire semaphore, sort `waiters` list in order of priority.
  - Modify `sema_down()` / `cond_wait()`

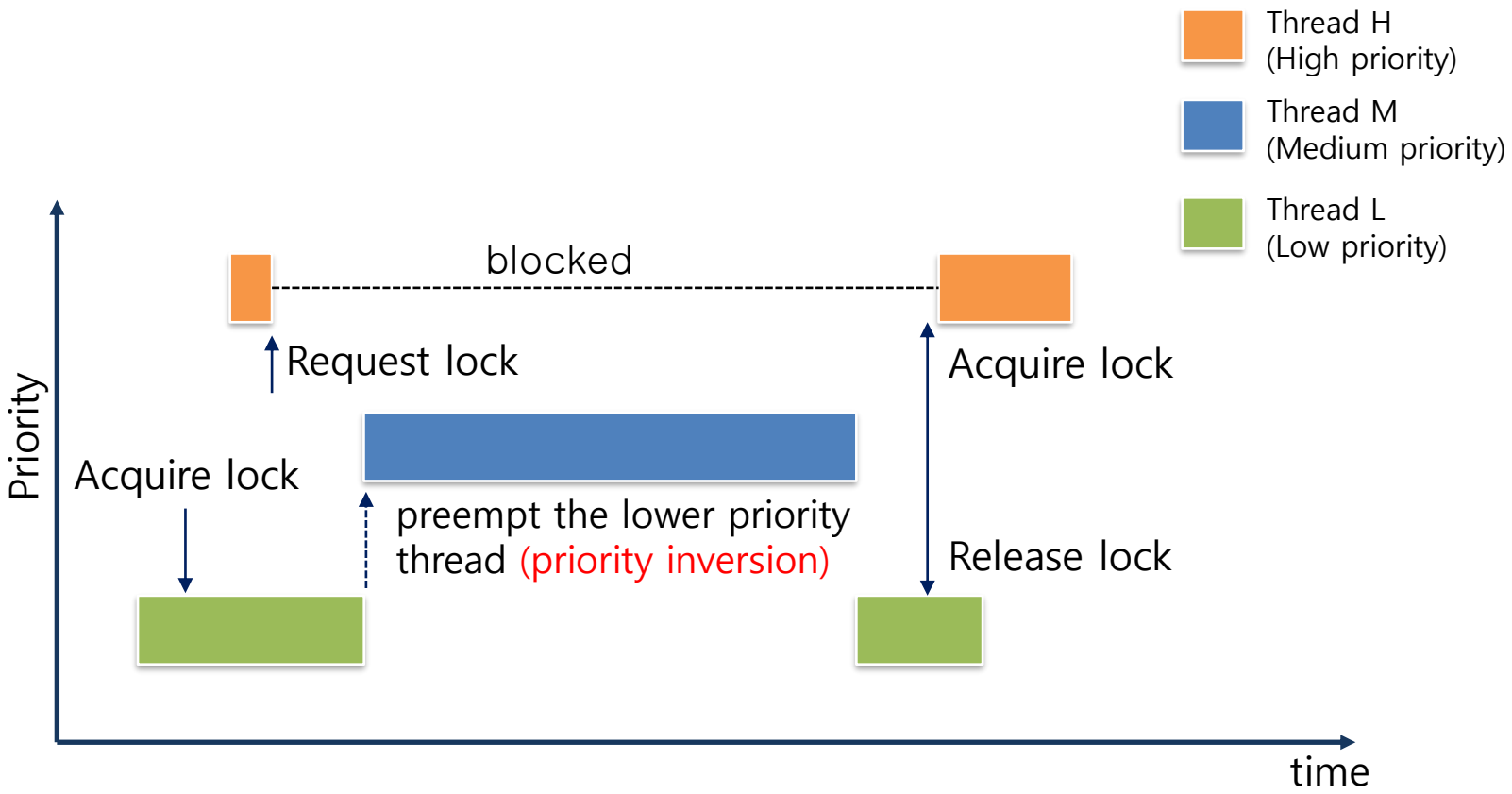






Priority Inversion

- The situation where thread of the higher priority waits thread of the lower priority.







Priority Inversion

In 1997, Pathfinder on Mars has stopped. OS has crashed due to the priority inversion.





- The Mars Pathfinder Mission Status Reports — First Week
- The Mars Pathfinder Mission Status Reports — Second Week
- The Mars Pathfinder Mission Status Reports — Third Week
- What really happened on Mars?
- A Conversation with Glenn Reeves

How did NASA remotely fix the code on the Mars Pathfinder?

17

5

In 1997, NASA remotely fixed a bug that caused priority inversion on their Mars Pathfinder. How did they go about doing this? What kind of communication protocols are used? How do they update the source for an operating system, compile it, and run it from a remote location? This might be simpler than I thought, but to me this seems like quite the feat!

Story of the bugfix here: http://research.microsoft.com/en-us/um/people/mbj/mars_pathfinder/authoritative_account.html

The author said to email him and he would provide details, but this was almost 20 years ago. Curious to see if anyone else knows how this worked.

asked

viewed

active

Linked

2



Priority Inversion

Sojourner “探路者” (PathFinder)

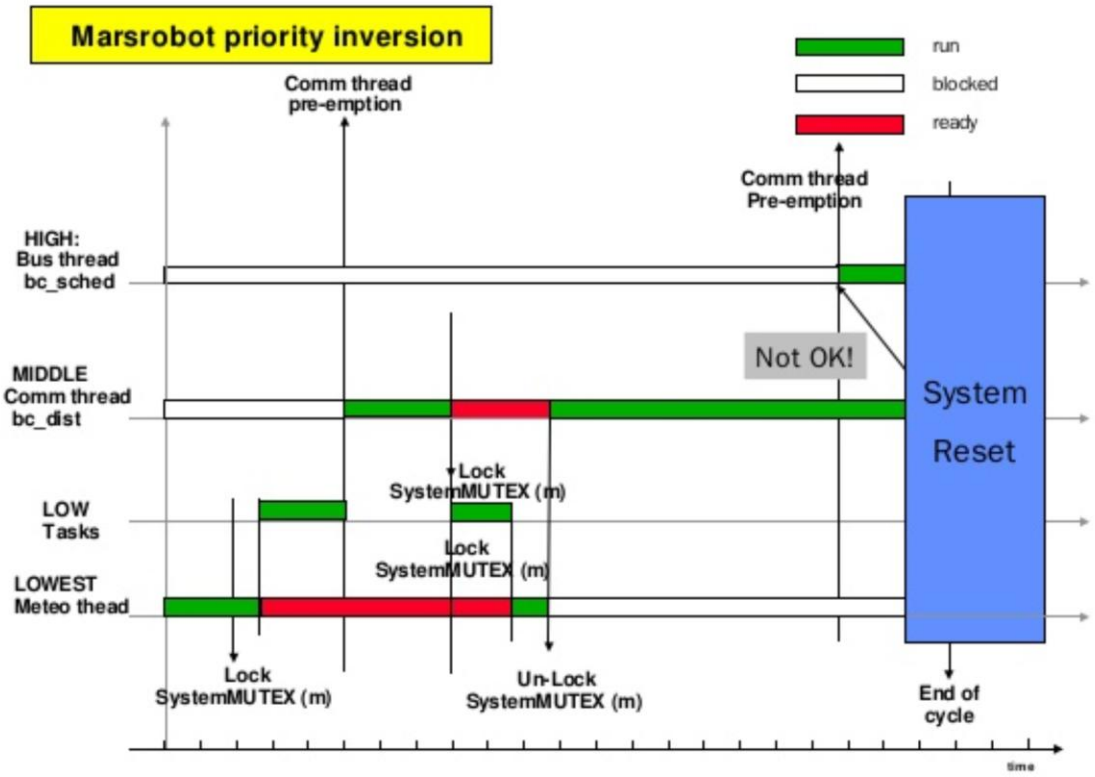


- Lander (登陆舱)
  - IBM Rad6000 SC (20 MIPS), 128 MiB RAM, 6 MiB EEPROM
  - VxWorks “实时” 任务操作系统
    - ASI/MET task: 大气成分监测 (低)
    - `bc_dist` task: 分发任务 (中)
    - `bc_sched` task: 总线调度 (高)
- Rover (火星车)
  - Intel 80C85 (0.1 MIPS), 512K RAM, 176K Flash SSD
- 着陆后开始出现系统重启





Priority Inversion



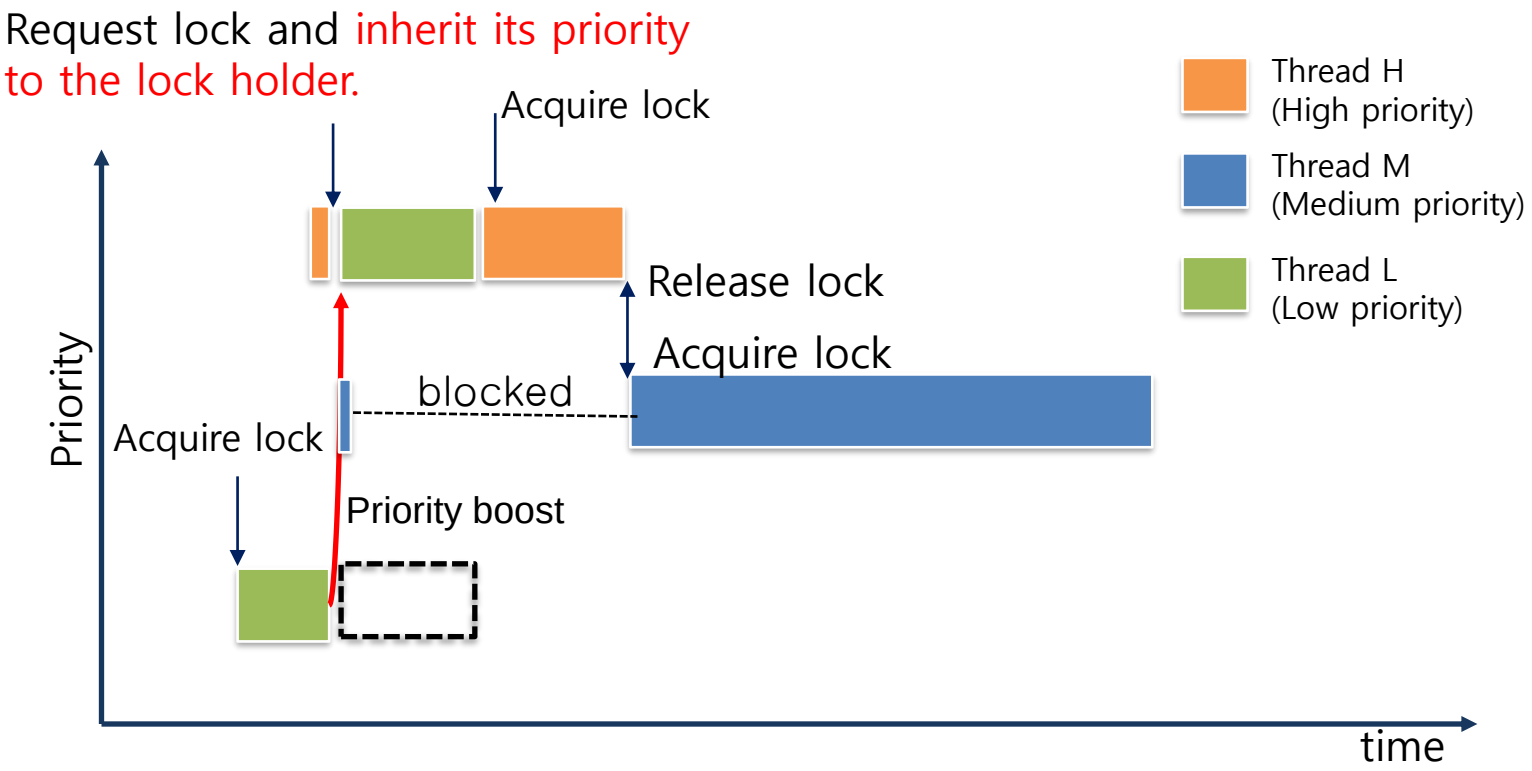
- (低优先级) `select -> pipeIoctl -> selNodeAdd -> mutex_lock`
- (高优先级) `pipeWrite -> mutex_lock`

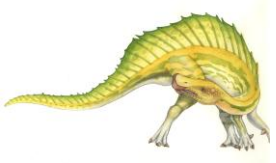




Priority Donation

- Inherit its priority to the lock holder.







Priority Inheritance · *Killing Priority Inversion*

UNSEEL.COM · HIGH · MEDIUM · LOW · 1 SHARED LOCK

HIGH WAITED 0 MS
BOOST NONE

unseel.com

State Three tasks · one shared lock

- High priority
- Medium priority
- Low priority
- Holds the lock

A MARS ROVER KEPT REBOOTING ITSELF 100...

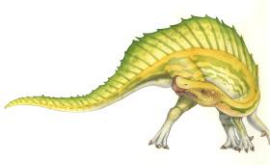


A Mars rover kept rebooting itself 100 million miles from Earth — because a low-priority task wouldn't let go of a lock. The fix had a name: priority inheritance.

1 / 9

Unseel.com · Priority Inheritance

▶ PLAY | ⏸ PAUSE | 🔊 UNMUTE | RESET | ARTICLE | VIDEO





Implementation of Priority Donation

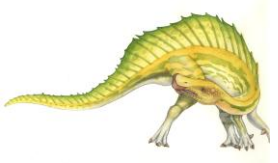
- Functions to modify
  - `static void init_thread(struct thread *t, const char *name, int priority)`
    - Initializes data structure for priority donation.
  - `void lock_acquire(struct lock *lock)`
    - If the lock is not available, store address of the lock.
    - Store the current priority and maintain donated threads on list (multiple donation).
    - Donate priority.
  - `void lock_release(struct lock *lock)`
    - When the lock is released, remove the thread that holds the lock on donation list and set priority properly.
  - `void thread_set_priority(int new_priority)`
    - Set priority considering the donation.





priority-* and mlfqs-*

- You can figure it out on your own :)
- Or get inspiration from other students' presentations



End of OS lab 3

